Project Title: Anti-Performance Throttling Booster for Applications and Gaming Systems

Project ID: 50

Team Name: Infinite Loop

Ankita Vellara	Ashritha Reddy	Harshitha Vangaveti	Hamsa M N
PES2UG24AM021	PES2UG24AM071	PES2UG24AM064	PES2UG24AM060

PROJECT TEST PLAN

0. Introduction

0.1 Purpose

The purpose of this Project Test Plan is to define the testing approach for the Anti-Performance Throttling Booster for Applications and Gaming Systems. The plan describes the test activities required to verify that the implemented software meets the functional and nonfunctional requirements specified in the Software Requirements Specification (SRS).

Testing will focus on real-time performance monitoring, performance-throttling detection, background-process analysis and optimization, network-usage monitoring, user-controlled performance profiles, alerts, recommendations, and event logging.

0.2 Objective

Build a Windows-oriented desktop tool that monitors CPU, memory, process and relevant network usage; detects conditions associated with performance degradation; presents alerts and recommendations; supports user-controlled performance profiles; and applies only permitted, user-confirmed optimization actions.

0.3 Scope

The testing scope includes the implemented features of the application, their interaction with one another, the correctness of displayed monitoring information, the handling of invalid or unavailable data, and the safety of optimization operations.
The application will be tested in the Windows-based desktop environment identified in the SRS.

Tests will also examine whether monitoring remains responsive, potentially disruptive actions require user confirmation, and critical operating-system processes are protected.

Testing will cover unit, integration, and system-level scenarios, together with manual verification of the application's user interface and behavior.

0.4 Functional Scope from the SRS

Feature area	Planned capability	SRS requirements
Real-time monitoring	Start/stop monitoring; CPU/RAM and process metrics; periodic refresh.	REQ-01–REQ-04
Throttling detection	Evaluate configured criteria; report suspected events with timestamps and relevant metrics.	REQ-05–REQ-08
Process analysis and optimization	List process usage; identify eligible background processes; require confirmation; report action outcome.	REQ-09–REQ-12
Network monitoring	Display usage statistics, associate them with the monitoring period and alert on configured thresholds.	REQ-13–REQ-15
Performance profiles	Provide predefined profiles, profile selection, supported edits and profile-aware decisions.	REQ-16–REQ-19
Alerts, recommendations and logs	Display relevant alerts and recommendations; log events/actions; show recent logs.	REQ-20–REQ-23

1. Lifecycle Model and Justification

Selected lifecycle: Agile iterative development with two prescribed implementation sprints. This model is selected because it supports Agile methodology, active backlog management and incremental delivery.

Reason for suitability: the monitoring, detection, profiles, optimization and logging modules can be developed and tested independently, then integrated incrementally. Early integration exposes interface issues before the final validation period. The process also supports individual contribution tracking through commits and pull requests.

2. Tools and Infrastructure

Purpose	Selected tool	Use in this project
Programming language	Python	Monitoring, detection, profile logic, optimization safeguards and logging.
Desktop GUI	Tkinter	Dashboard, controls, metric displays, alerts and log view
System metrics	psutil	Supported CPU, memory, process and network statistics.
Version control	Git	Branches, commits and merges.
Repository / collaboration	GitHub	Repository, pull requests, Issues and code review.
Agile planning	Jira	Backlog, story points, assignments, sprint tracking and status.
CI	GitHub Actions	Run tests and configured quality checks on pushes and pull requests.
CI/CD tool specified	Jenkins	Continuous Deployment
Unit testing	pytest	Tests for monitoring, detection, profiles, logging and safeguards.
Coverage	pytest-cov	Measure and report test coverage; review branch coverage where supported.
Static analysis	SonarQube	Code-quality and maintainability checks.
Containerization	Docker	Reproducible test/build environment where applicable; not required for desktop runtime itself.
Design/documentation	UML, Markdown, PDF	Architecture, use cases, module design, test evidence and final report.

3. Deliverables: Reuse and Build Classification

Deliverable/component	Classification	Justification
Python runtime and standard libraries	Reuse	Existing language/runtime functionality.
Tkinter	Reuse	Provides GUI widgets; project-specific dashboard and interactions are built.
psutil	Reuse	Provides supported operating-system metrics and process information.
Git, GitHub and GitHub Projects	Reuse/configure	Existing source-control, collaboration and backlog infrastructure.
GitHub Actions / Jenkins	Reuse/configure	Existing automation platforms.
pytest, pytest-cov, static analysis tools	Reuse/configure	Existing quality tools; the team builds test cases and configures checks.
Monitoring engine and refresh workflow	Build	Project-specific coordination, sampling, and presentation of metrics.
Throttling-detection logic	Build	Project-specific criteria and suspected-event classification.
Process eligibility and safety checks	Build	Project-specific protection rules and confirmation workflow.
Profile management and optimization controller	Build	Project-specific profile settings, guarded actions and outcome reporting.
Network metrics presentation	Build	Project-specific display, thresholds and monitoring-period association.
Alerts, recommendations and event/action logger	Build	Project-specific rules, timestamps and history view.
Tkinter dashboard and module integration	Build	Project-specific screens, controls and integrated user workflow.
Test suite, traceability matrix and final report	Build	Team-produced tests, actual results, evidence and requirement mappings.
Sprint videos and demonstration material	Build	Team-produced evidence of working sprint outcomes.

4. Work Breakdown Structure (WBS)

The WBS decomposes the project into manageable work packages. Effort is estimated in person-days (PD). The baseline totals 140 PD, equivalent to 7.0 person-months assuming 20 working days per person-month.

No.	Work package	Principal tasks / output	Effort (PD)
1	Project planning and requirements	Scope, SRS mapping, backlog, acceptance criteria, story points, risk list.	8
2	Architecture and design	Module interfaces, architecture/use-case diagrams, data structures, UI flow and safety rules.	10
3	Monitoring and process analysis	Start/stop monitoring, CPU/RAM sampling, process listing and resource usage.	22
4	Throttling detection	Detection criteria, suspected-event classification, timestamps and metric context.	12
5	Profiles and optimization	Predefined profiles, supported settings, confirmation, permission checks and action results.	18
6	Network, alerts and logging	Network usage statistics, threshold alerts, recommendations, event/action logs and log view.	14
7	GUI and integration	Tkinter dashboard, controls, module integration and error/status presentation.	16
8	Testing and validation	Unit, integration and system tests; negative tests; defect fixes; traceability and coverage.	22
9	CI and code quality	GitHub Actions, test automation, coverage reporting and static/security checks.	8
10	Documentation and release	Final report, screenshots, test results, sprint videos, presentation and release freeze.	10
	TOTAL	Baseline estimate	140 PD / 7.0 PM

5. Effort Estimation

Person-month estimate = estimated person-days ÷ 20 working days.
The 140 PD total is a rough effort estimate for all team members combined.

Activity group	Effort (PD)	Effort (PM)
Requirements and planning	8	0.40
Architecture and design	10	0.50
Monitoring and process analysis	22	1.10
Throttling detection	12	0.60
Profiles and optimization	18	0.90
Network, alerts and logging	14	0.70
GUI and integration	16	0.80
Testing and validation	22	1.10
CI and code quality	8	0.40
Documentation and release	10	0.50
TOTAL	140	7.0

Capacity check: Four team members × five working days × eight weeks = 160 person-days of theoretical capacity.
The 140 PD estimate leaves 20 PD of capacity for coordination, integration delays and unplanned defects.

6. Gantt Chart

